# Your final target after 4 days

By the end of Day 4, you should have these finished:

### Graphics side

- one playable rescue map
- terrain sculpted
- textures applied
- trees, rocks, logs placed
- lighting and atmosphere added
- NavMesh baked

### Intelligent Systems side

- grid-based graph generated from the world
- each node marked walkable or unwalkable
- adjacency list created
- graph data available to Student 3 and Student 4
- function ready for Student 2 to update nodes when logs move

### Proof for viva

- you can explain what a node is
- you can explain how you detect blocked cells
- you can explain how neighbors are created
- you can explain why your graph design is efficient enough for this project

---

# Important rule for your 4 days

Do **not** try to make a large or very realistic jungle.

Make:

- one medium terrain
- one start area
- one goal area
- one blocked path
- one alternate path or small side route
- a few trees and rocks
- 2 or 3 key obstacle logs

That is enough to get marks and easier to finish.

---

# DAY 1

# Build the world foundation

## Main goal

Finish the **basic environment** and prepare it so you can later convert it into a graph.

## What you should complete today

### 1. Decide the map layout on paper first

Before opening Unity, sketch a very simple layout.

Your map should have:

- drone start position
- rescue goal or beacon area
- main path
- blocked section with fallen logs
- alternate route or open area
- trees and rocks that act as natural blockers

Keep it small. A compact map is easier to graph.

### 2. Create the terrain

In Unity:

- create the terrain
- sculpt gentle hills only
- avoid steep mountains
- keep large flat or slightly uneven areas for pathfinding

Do not overdo terrain shaping. Pathfinding becomes harder on messy terrain.

### 3. Paint terrain textures

Apply 3 to 4 textures only:

- grass
- mud
- dirt path
- rock or dry soil

Use them consistently:

- dirt on paths
- grass around open areas
- mud near storm-damaged areas
- rock on blocked or rough areas

## 4. Place environment objects

Add:

- trees
- rocks
- logs
- bushes if needed

Place them for gameplay, not just beauty.
You need some cells to become blocked because of these objects.

## 5. Create clear gameplay zones

Mark these clearly:

- **Start/Base**
- **Blocked path**
- **Goal/Rescue beacon area**

These zones help everyone else in the team.

## 6. Set up atmosphere

Add simple mood:

- cloudy skybox if available
- slight fog
- softer or darker lighting
- maybe a storm-like feel

Keep it light enough so the map is still visible.

## 7. Organize the hierarchy

Do not keep random objects everywhere.
Create folders or parent objects like:

- Terrain
- Environment
- Trees
- Rocks
- Obstacles
- StartPoint
- GoalPoint
- GraphSystem

This helps for cleanliness and viva.

---

# Day 1 deliverables

By tonight you should have:

- one complete rough map
- textures applied
- obstacles placed
- lighting added
- clear start and goal positions

---

# Day 1 Git commits

Make 3 to 5 commits, not one giant commit.

Examples:

- `Create base terrain and level layout`
- `Add jungle props and blocked route`
- `Apply terrain textures for rescue zone`
- `Set up basic storm lighting and fog`
- `Organize scene hierarchy for world builder role`

---

# Day 1 coordination with teammates

Tell Student 2:

- which objects are intended to be movable logs

Tell Student 3:

- start and goal positions
- approximate playable area size

Tell Student 4:

- path area where drone will fly

---

# DAY 2

# Build the graph system

## Main goal

Convert the world into a **custom mathematical graph**.

This is your most important IS work.

## What you should complete today

### 1. Decide the graph structure

Use a **grid-based graph**.
This is the easiest and safest.

Each node should store:

- grid x
- grid y
- world position
- isWalkable
- list of neighbors

Optional:

- movement cost

### 2. Decide the grid size

Choose a reasonable cell size.
For example:

- 1 unit

- 1.5 units
- 2 units

Do not make it too tiny or too huge.

If it is too tiny:

- too many nodes
- more debugging

If too huge:

- path becomes inaccurate

### 3. Create a `Node` class

You should make a script for node data.

It should include:

- grid coordinates
- world position
- walkable state
- neighbors

Keep it clean and simple.

### 4. Create a `GridManager` or `GraphBuilder`

This script should:

- define graph width and height
- loop through grid positions
- cast downward raycasts
- detect if ground is present
- detect if obstacle is present
- mark node as walkable or unwalkable

### 5. Use raycasts or overlap checks

A simple method:

- cast ray down from above each grid cell
- if it hits terrain, that location exists
- then check whether tree/rock/log blocks it
- if blocked, mark unwalkable

You can use layers for:

- Ground
- Obstacle

That makes checking easier.

### 6. Build adjacency list

After nodes are created:

- connect each walkable node to valid neighbors

Simplest choice:

- 4 directions only

If you want safer completion in 4 days, use 4-direction movement.
It is much easier to debug.

### 7. Add a debug print test

Print:

- total nodes
- total walkable nodes
- total blocked nodes
- neighbor count for sample nodes

This helps prove your graph works even before full 3D integration, which the IS brief cares about.

### 8. Add temporary visual debug

Even before Student 4 does full debug mode, you can help yourself by drawing:

- green gizmos for walkable nodes
- red gizmos for blocked nodes

This is very useful for testing.

---

# Day 2 deliverables

By tonight you should have:

- node class completed
- graph/grid generated over the map
- walkable and blocked cells detected
- neighbors connected

- graph visible in Scene view or debug logs

---

## Day 2 Git commits

Examples:

- `Add Node data structure for custom graph`
- `Implement grid generation over terrain`
- `Add walkable and blocked node detection`
- `Create adjacency list for neighboring nodes`
- `Add gizmo debug for graph visualization`

---

## Day 2 coordination with teammates

Tell Student 3:

- how to access nodes
- how to get neighbors
- how start and goal nodes can be found

Tell Student 2:

- how blocked logs currently affect graph
- what function they should call later to update nodes

---

# DAY 3

# Make your system usable by the team

## Main goal

Turn your graph into a clean system that other members can plug into.

## What you should complete today

### 1. Add public access methods

Your graph system should expose useful functions like:

- get node from world position
- get neighbors of a node
- check if node is walkable
- mark node walkable/unwalkable
- rebuild a small part of graph if needed

This is what makes your work actually usable.

## 2. Add node update support

Student 2 will need this later.
Create a method like:

- `UpdateNodeWalkabilityFromWorldPosition(...)`
- or `SetNodeWalkable(x, y, bool)`

This is very important because your proposal already says Student 2 will tell your graph to update a node when a log moves.

## 3. Connect graph with actual obstacle positions

Test with one fallen log:

- when log is placed on the route, certain nodes are blocked
- when moved away manually in editor, graph updates correctly after refresh or method call

You do not have to fully finish Student 2's event system, but your side must be ready.

## 4. Bake initial NavMesh

Since your GV role includes the initial NavMesh, do it now.

Even if the AI uses custom graph, you should still bake NavMesh because:

- it is part of your graphics role
- it shows proper environment setup
- it helps explain world readiness

## 5. Improve lighting and environment quality

Now polish the world a bit:

- fix ugly texture stretching
- adjust shadows
- improve placement of trees and rocks
- make blocked area visually obvious

### 6. Remove unnecessary clutter

Optimization matters in GV.
So:

- remove extra heavy assets
- avoid too many high-detail props
- keep scene clean

### 7. Document your graph logic for viva

Write a small note for yourself:

- node = one grid cell in world space
- edge = connection to adjacent walkable nodes
- adjacency list = neighbor storage
- blocked objects create unwalkable nodes
- graph is generated programmatically with raycasts

You will need this.

---

## Day 3 deliverables

By tonight you should have:

- graph system ready for teammates
- update functions created
- NavMesh baked
- environment cleaned and improved
- one test showing graph reacts correctly to obstacle position changes

---

## Day 3 Git commits

Examples:

- `Expose graph API for pathfinding integration`
- `Add node update functions for dynamic obstacles`
- `Bake initial NavMesh for world layout`
- `Refine terrain props and blocked route visuals`
- `Optimize scene hierarchy and asset placement`

---

## Day 3 coordination with teammates

Sit with Student 3 and show:

- how to get start node and goal node
- how neighbors are returned
- what data structure they should use

Sit with Student 2 and show:

- which update method they should call when logs move

If possible, give both of them a simple usage example.

---

# DAY 4

# Final integration, testing, and viva preparation

## Main goal

Finish strong. Make your work stable, presentable, and explainable.

## What you should complete today

### 1. Full graph test

Test these cases:

- open path
- blocked path
- mixed path with obstacles
- edge cells
- corners of map

Check for:

- null references
- missing neighbors
- wrong blocked nodes
- graph cells outside terrain

## 2. Test with Student 3's A*

If their A* is ready:

- give them your graph
- test path output
- make sure node positions are correct
- make sure path does not go through blocked objects

This is important because the IS group marks reward cohesion between graph, searches, recalculation, debugger, and graphics environment.

## 3. Test with Student 2's interaction logic

If their interaction is ready:

- move one log
- update corresponding nodes
- confirm graph state changes correctly

Even one successful demo case is enough.

## 4. Final polish on world builder role

Improve:

- lighting balance
- texture blending
- tree placement
- blocked area readability
- start and goal visual clarity

Your world should not look random. The GV sheet clearly values visual cohesion and proper level layout, lighting, texturing, and NavMesh baking.

## 5. Clean your scripts

Before finishing:

- rename messy variables
- add comments
- remove dead code
- split large methods if needed

The IS sheet explicitly values clean, modular code and role-specific commits.

## 6. Prepare your viva explanation

You should practice answering these:

**What is your role?**
I handled level design, lighting, texturing, initial NavMesh, and custom graph formulation.

**How does your graph work?**
I divided the terrain into a grid. Each cell becomes a node. I used raycasts and obstacle checks to decide whether a node is walkable. Walkable nodes are connected to neighbors and stored in an adjacency list.

**Why did you choose a grid graph?**
It is simple, reliable, easy to visualize, and works well for this project within the time and scope.

**What happens when an obstacle changes?**
The affected nodes can be updated from unwalkable to walkable, or the reverse, and that updated graph is then used by the search algorithm.

**What did you do for graphics?**
I created the map layout, textured terrain, placed environment props, added lighting and fog, and baked the initial NavMesh.

### 7. Capture proof

Take screenshots or short clips of:

- scene layout
- graph gizmos
- blocked vs walkable nodes
- terrain and lighting
- NavMesh view if possible

These are useful for presentation and defense.

---

# Day 4 deliverables

By tonight you should have:

- finished world builder visuals
- working graph generation
- usable graph API
- tested graph with team systems
- clean code
- viva explanation ready

---

# Day 4 Git commits

Examples:

- `Fix graph edge cases and null checks`
- `Integrate graph data with A star input`
- `Polish level lighting and texture blending`
- `Clean graph builder code and comments`
- `Finalize student 1 world and graph systems`

---

# What you must not waste time on

Do not spend your 4 days on:

- making a huge map
- ultra realistic terrain
- advanced shaders
- too many asset details
- perfect cinematic lighting
- complex procedural generation
- fancy UI

Your marks come from **clear role completion**, not from making the biggest game.

---

# Minimum version that still gets good marks

If time becomes very short, finish this minimum version:

### Graphics minimum

- one medium terrain
- 3 terrain textures
- trees and rocks placed
- one blocked route
- one goal area
- simple lighting and fog
- initial NavMesh baked

### IS minimum

- graph generated on terrain

- walkable/unwalkable detection works
- adjacency list works
- graph can be used by A*
- at least one node update method exists for moved obstacles

If that is stable, your part is already solid.

---

# Best order inside each day

Use this structure every day:

### Morning

- build one core feature

### Afternoon

- test and fix it

### Evening

- clean scene or code
- commit your work
- send update to group

That will keep you from getting stuck.

---

# Your personal checklist for the 4 days

Tick these off one by one:

- terrain created
- textures applied
- props placed
- lighting added
- blocked route designed
- start and goal marked
- node class created
- grid generated
- blocked/walkable checks done
- adjacency list built

- gizmo debug added
- update methods added
- NavMesh baked
- integrated with teammates
- cleaned code
- viva explanation prepared